Massachusetts Institute of Technology - Introduction to Algorithms (6.1210)

Xavier Henry

September 2026

Contents

1 Lecture 1 . . . . . **4**
1.1 Integer Addition 4
1.2 Integer Multiplication 4
1.2.1 Divide and Conquer 4
1.2.2 Karatsuba's Algorithm 5

2 Lecture 2: Recursion **5**
2.1 Sorting 5
2.1.1 Selection Sort 5
2.1.2 Peak Finding 7

3 Lecture 3: Divide and Conquer, Iteration **7**
3.1 Merge Sort 8
3.1.1 Proof of Correctness 8
3.2 Inversion Counting 9
3.3 2 Dimensional Peak Finding 9
3.3.1 Divide and Conquer 10

4 Lecture 4: Reduction and Randomization **10**
4.1 Review 10
4.2 Another Loop: Polish Flag 10
4.3 Reductions 11
4.3.1 Types of Reduction: Turing/Oracle and Many-one 11
4.3.2 Median Reduces to Selection (Many-One) 12
4.4 Randomisation 13
4.4.1 Quick Select 13

5 Lecture 5: Sequences **14**
5.1 Stack Interface and its Implementations 14
5.1.1 Linked List 14
5.1.2 Static Array 15
5.1.3 Dynamic Array 15
5.2 Amortized Analysis 15

5.3 Memory Overhead . . . 16

6 Lecture 6: Sets and Hashes . . . 16
6.1 Comparison Model of Computation . . . 16
6.1.1 Division Method . . . 17
6.1.2 Universal Hash Function . . . 17

7 Lecture 7: Radix Sort . . . 18

8 Lecture 8: Binary Search Trees . . . 18
8.1 Traversals . . . 18
8.1.1 In-Order Traversals . . . 19
8.1.2 Pre-Order Traversal . . . 19
8.1.3 Post-Order Traversal . . . 19
8.2 Binary Search Trees and Sets . . . 19

9 Lecture 9: Augmented Binary Trees . . . 19
9.1 Sequences as Binary Trees . . . 19
9.2 Tree Augmentations . . . 19
9.3 Subtree Size: Order Statistics and Sequences . . . 20
9.4 Determining the Rank of an Element . . . 20

10 Key Formulae and Theorems... Thanks ChatGPT . . . 21
10.1 Theorems to Know . . . 21
10.1.1 Master's Theorem . . . 21
10.1.2 Fermat's Little Theorem . . . 21
10.1.3 Bezout's Identity . . . 21
10.1.4 Euclidean Algorithm . . . 21
10.1.5 Extended Euclidean Algorithm . . . 21
10.1.6 Euler's Theorem . . . 22
10.1.7 Chinese Remainder Theorem . . . 22
10.1.8 Binomial Theorem . . . 22
10.1.9 Pascal's Identity . . . 22
10.1.10 Handshaking Lemma . . . 22
10.1.11 Pigeonhole Principle . . . 23
10.1.12 Inclusion–Exclusion Principle . . . 23
10.1.13 Cauchy–Schwarz Inequality . . . 23
10.1.14 Triangle Inequality . . . 23
10.1.15 Law of Total Probability . . . 23
10.1.16 Bayes' Theorem . . . 24
10.1.17 Linearity of Expectation . . . 24
10.1.18 Markov's Inequality . . . 24
10.1.19 Chebyshev's Inequality . . . 24
10.1.20 Union Bound . . . 24
10.1.21 Geometric Distribution . . . 24
10.1.22 Birthday Bound . . . 25
10.2 Summation Formulae . . . 25
10.2.1 Basic Summations . . . 25

10.2.2 Arithmetic Series: 25
10.2.3 Geometric Series: 25
10.2.4 Harmonic Series: 26
10.2.5 Logarithmic Summations: 26
10.2.6 Powers of Two: 26
10.2.7 Telescoping Sums: 26
10.2.8 Useful Rules: 27
10.2.9 Integral Approximation: 27

1 Lecture 1

Definition 1: Algorithm

An algorithm is a step by step procedure for solving a computational, problem. A computational problem takes inputs, and for each input, outputs one or more resulting outputs.

1.1 Integer Addition

Consider the preschool addition algorithm $x + y$ where we add 1 to x, y times. The running time of this is $\Theta(10^n)$. Prove this!
Our grade school algorithm, wherein we line up the digits and add (assuming we've memorized the single-digit addition table), takes linear time, $\Theta(n)$

Theorem 1 : Integer Addition Lower Bound

Every correct algorithm for integer addition requires $\Omega(n)$ time.

Proof. Our output must have at least n digits. □

Alternatively, consider the following (more subtle) proof

Proof. For our algorithm to be correct, we must read every input digit, of which there are $\Omega(n)$. Suppose we have a magical correct addition algorithm f that does not read every input digit. Without loss of generality, suppose the digit that is not touched is x_0. There exists x, y and x', y' which differ on only x_0 st. $x + y \neq x' + y'$ but $f(x, y) = f(x', y')$. □

1.2 Integer Multiplication

We now consider the preschool integer multiplication, where we add x to itself y times. This is exponential time.
Consider also the grade school multiplication algorithm. This is the standard algirithm of lining up the digits and multiplying them with our multiplication tables. We know that each digit in x must touch each digit in y, so this must be $\Theta(n)$.
Importantly, is there a better algorithm that is correct, yet faster?
We know the size of our output is linear $O(n)$ and the size of our input is $O(n)$.

1.2.1 Divide and Conquer

A divide and conquer algorithm divides our problem into subproblems, recursively solves each subproblem, and assebles into a solution. We can use such an algorithm for integer multiplication (figure this out)!

$$
\begin{aligned}
A &= mult(x_l, y_l) \\
B &= mult(x_h, y_l) \\
C &= mult(x_l, y_h) \\
D &= mult(x_h, y_h) \\
T(n) &= \text{worst-case time on n-digit inputs} \\
T(n) &\leq 4T(n/2) + Cn \\
T(n) &\leq O(n^2)
\end{aligned}
$$

1.2.2 Karatsuba's Algorithm

Instead, let's do:

$$
\begin{aligned}
A &= mult(x_l, y_l) \\
D &= mult(x_h, y_h) \\
E &= mult(x_l + x_h, y_l + y_h)
\end{aligned}
$$

And our output: $A + 10^{n/2}(E - A - D) + 10^n D$. This becomes clear if we FOIL.
Now we prove our running time.

Proof.

$$
\begin{aligned}
T(n) &\leq 3T(n/2 + 1) + Cn \\
T(n) &\leq n^{\log_2 3} \\
&= O(n^{1.6})
\end{aligned}
$$

□

With some analysis of the algorithm, we can prove that it is tight.
It's important to note, that though Karatsuba is better asymptotically, for smaller input values, the speed vs. grade school multiplication depends heavily on our implementation of the algorithm.

2 Lecture 2: Recursion

2.1 Sorting

Input: Array A
Output: Permutation σ such that $A \circ \sigma$ is monotone.

2.1.1 Selection Sort

We will analyze a recursive implementation of selection sort.

Proof. This is just the implementation. Base case: If $|A| \leq 1$ return A
Otherwise: Let $m :=$ Arg ,min A. Swap $A[0], A[m]$
Call selection sort on $A[1:]$ □

You can see this is very similar to induction. We now write a proof of correctness

Proof of Correctness Correctness: Inductive Preidcate $P(n)$, Alg correct on size n input.

Proof. Base Case: If $|A| \leq 1$ already sorted, so return A (correct result).
Inductive Step: Suppose $|A| > 1$ and $P(n-1)$. We want to prove that we end up with a permutation of A we call A' (1) and that this permutation is monotone (2). To prove (1), we ensure that at each step, we are applying a permutation. Finding the min of A has no write, performing a swap is a transposition and $SS(A[1:])$ permutes by our inductive hypothesis, since we assume P for smalller values. Therefore, A' is a permutation of A. (proved 1)
We swap the min of A to $A'[0]$. By our inductive hypothesis, $A'[1:]$ is monotone. $A'[1]$ is in A so is no smaller than min A.
$A'[0] \leq A'[1] \leq ...$ so A' is monotone (proved 2)
By induction, our implementation of selection sort is correct. □

Efficiency and Proving Bounds Quantify input

Definition 1: Worst-Case Runtime

The worst-case runtime $T(n)$ of A is our maximum over all x of size n of number of opperations used by $A(x)$. A has worst case runtime $O(f)$ means for sufficiently large x, $A(x)$ uses $\leq$ constant factor for more than $f(|x|)$ operations. Finding this worst-case runtime gives us a lower bound.
If we want to prove an upper bound on our worst-case runtime, we want to prove that $A(x)$ always takes $O(f(|x|))$ time. As said before, to prove a lower bound we need **one** input of each length for which $A(x)$ is slow. If we choose a particularly fast input for A to lower bound our algorithm, this is wrong.

Definition 2: Model of Computation and Atomic Operations

A model of computation is an abstraction of a computer. We use what is called the word-RAM model, which closely mimicks the operations of an actual computer. We use w bit words as a unit of memory. Our atomic operations are $+, -, *, /, etc.$, AND, OR, NOT, comparisions, ... Any bianry operation is atomic. We can read and write from/to specific memory addresses. We also have control flow, so things like IF, WHILE, RETUR. We can do all these atomic operations in constant time.

To analyze the runtime of selection sort, we need a recurrence relation.

$$T(n) \in \Theta(1) \text{if} n \leq 1$$
$$T(n) \leq \Theta(n) + \Theta(1) + T(n-1)$$
$$T(n) \leq O(n^2)$$

We ignore the largest half of elements $n/2$ times.
Lower bound: $n/2$ elements read $n/2$ times each $\Omega(n^2)$. Say we have a monotonically decreasing list that we want to make monotonically increasing. By the time selection sort reaches $\lfloor n/2 \rfloor$, we have sorted the first half of the list, and swapping has sorted the second half of the list. This proves our n^2 lower bound, since we've done at least $n/2$ work for $n/2$ elements.

2.1.2 Peak Finding

Definition 3: Peak

A peak of array A is an index i such that $A[i]$ is at least as large as its neighbours (local maximum)

Peak-finding Problem Input: Array A (non-empty)
Output: Peak of A (any)
Edge cases of input should be considered carefully in the problem definition!

Algorithm 1: Exhaustive Search As counter intuitive as this may seem, this is a perfectlty valid algorithm definition.
We first prove correctness.

Proof. Correctness by exhaustion. □

Runtime:

Proof. Upper: By definition we do $\Theta(1)$ check at most $O(n)$.
Lower: Suppose A strictly monotone. Our check fails $\Omega(n)$ times. (notice here we picked a super slow input and used that as our lower bound!) □

Algorithm 2: Binary Search Recursion: discard half (constant fraction) of input quickly.
Idea: check the midpoint (if peak, then return midpoint), go uphill

Proof. Here's our implementation:
PF(A): If length 1 outpit 0
Otherwise, Let $j = \lfloor n/2 \rfloor$. If peak output j. If $A[j-1] \geq A[j]$ recurse on $A[:j]$. Otherwise, recurse on $A[j+1:]$. □

Correctness

Proof. Base case: if singleton index 0 is peak vacuously.
Inductive case: Suppose $n > 1$, PF is correct on inputs smaller than n.
Case 1: j is peak. Correctly return j.
Case 2: Assume without loss of generality $A[j-1] \geq A[j]$.
WTP: returned peak of $A[:j]$ is also a peak in A.
Case 1: $k < j - 1$ then neighbours are same, so still peak.
Case 2: Case 1 + guard???
□

TODO: runtime analysis.

3 Lecture 3: Divide and Conquer, Iteration

- divide
- conquer - solve all of the subproblems recursively
- recombine - recombine all the solutions to subproblems together to get our final solution

3.1 Merge Sort

Divide input into two halves. Recursively sort both halves then marge sorted subarrays into a sorted array. Let's make a problem statement

Proof. MERGE
INPUT: two arrays A, B
PRECONDITION: $A + B$ sorted
OUTPUT: an array C which is the sorted concatenation of A and B
Initialize two indices i, j. Initialize empty C of size $2n$.
While $i + j < 2n$:
{
if $A[i] \leq B[j]$ then $C[i + j] := A[i]$, increment i
otherwise, $C[i + j] := B[j]$, increment j
}
Output C □

3.1.1 Proof of Correctness

We could try to use induction, but this would cause us something called buildup error. That is, with induction it's not clear whether we are actually building toward an answer. Here, it's best to use our **invariants**! But how do we do this?

- Model loop as state machine
- Think of a **state** as a snapshot of memory
- Start state: initialization
- transitions: executions of loop body
- final states: loop **guard** (What is a loop guard??? -¿ it's just the while loop condition lol)

When doing a proof with invariants, we do not necessarily want to prove correctness directly. Instead, we want to prove partial correctness. That is, if the algorithm terminates, it produces the correct answer. So what do we need for our invariant proof?

- Inductive predicate: Choose correct invariant(s)
- Initialization: Prove that the invariant(s) are true at initialization (our start state)
- Preservation: we need to prove that our invariants are preserved properties throughout the operation of our state machine.
- Prove invariants $\implies$ partial correctness
- Runtime analysis (proves termination)

We first prove partial correctness of our **merge** algorithm (putting the two lists together).

Proof. **Partial Correctness of Merge** A, B read only
I5: $C[: i + j]$ is the concatenation of $A[: i] + B[: j]$
I6: $C[: i + j]$ is sorted
I3: $A[i] \geq C[: i + j]$

I4: $B[j] \geq C[:i+j]$
I1: $0 \leq i \leq n$
I2: $0 \leq j \leq n$
Initialization: All vacuous since $i = j = 0$
Preservation: WLOG $A[i] \leq B[j]$
I3: WTP $A[i+1] \geq C[:i+j+1]$
By precondition, $A[i+1 \geq A[i] \geq C[i+j]$
I4: Because $B[j] \geq A[i]$, as above
I5: Adding $A[i]$ to both prefix of A and of C
I6: follows from I3,I6
Partial Correctness: Follows from I5, I6, and the loop guard. When our loop guard fails, C will be sorted and will be a permutation of all of A and all of B. □

In order to prove our merge sort algorithm (recurisve), we need to use strong induction. Prove this later

Proof. Strong induction. □

Let's do runtime analysis now!

Proof. $i+j$ starts at 0, increments with every loop body $\leq 2n$ by loop guard, so $2n$ iterations $\Theta(1)$ per $\Theta(n)$ total.

$$T(n) \leq 2T(n/2) + O(n)$$
$$T(n) \leq O(nlogn)$$

and we have our upper bound. Now, for our lower bound. Take any valid input of length n. Every element is merged $\Omega(logn)$ times and our merge algorithm takes linear time so $\Omega(nlogn)$. □

3.2 Inversion Counting

Inversion: pair (i, j) such that $i < j$ and $A[j] < A[i]$.
Alg1: Brute force, Runtime $\Theta(n^2)$, Correctness: exhaustive.
But notice that inversion counting is roughly how sorted is A? In fact, it's an interesting problem to sort our array, then count the number of inversions. In this case, making our problem a bit harder makes our ideal algorithm much faster.
Idea: Suppose $A = A[:m] + A[m:]$ either $i < j < m$ (or $m \leq i \leq j$ which we can count recursively) or $i < m \leq j$
While merging, if we take from right half, add $m - i$, i being our iteration index.

3.3 2 Dimensional Peak Finding

TODO: This was taught in one of the (very old) 6.006 lectures! You should totally watch his coverage of this. Input: $n \times m$ array A.
Output: any peak in A

Exhaustive Search : $\Theta(nm)$ time (Is this linear?)

3.3.1 Divide and Conquer

Look at mid col j. Find a peak in j, then go uphill. Let's find a peak that is the maximum in its column (this prevents an edge case with our boundary).

Runtime :

$$\begin{aligned} T(m,n) &\leq T(m,n/2) + \Theta(m) \\ &\leq O(mlogn) \end{aligned}$$

Take $N := mn$ then it is $O(\sqrt{N}\log N)$

4 Lecture 4: Reduction and Randomization

4.1 Review

Divide and conquer; three techniques:

- Loop invariants: we start with an initial state, and as long as a loop guard held true, we stay in the loop and change things (while maintaining our invariants, which we prove) and give an output.
- Make the problem harder, so that we move from a specific case of something to a more general problem that is easier to analyze and/or solve
- Recurrences - **Master's Theorem!** You should ~~probably~~ DEFINITELY memorize this!

4.2 Another Loop: Polish Flag

Problem Given an array A of n balls, each of which is either red or white, rearrange them in-place so that all the red balls are to the left.

Algorithm Pretend that we have two elements, $A[-1]$ (this is the thing in the far left, not the last element like in Python) as a dummy white ball, and $A[n]$ as a dummy red ball. We fix two indices: $w = -1, r = n$. Notice we start right before the first elem and right after the last elem, respectively. Why do we need these dummies though?
While $w \leq r$: swap $A[w] \iff A[r]$, increment w until $A[w]$ is white, decrement r until $A[r]$ is red.
Invariants:

- I1: $A[r]$ is red
- I2: nothing to the right of $A[r]$ is red (we want whites to the right... this is such a funny phrase it made me chuckle please don't cancel me I'm just trying to make my notes fun so I get an A)
- I3: $A[w]$ is white
- I4: nothing to the left of $A[w]$ is white (we want reds to the left)

Correctness

Proof. The first idea of the correctness proof is showing that our entire operation permutes the array. That is, our only whites are swaps $\implies$ permutation. Thus, we don't have to consider resizing or any alterations to the array, only proving the reds are to the left and the whites to the right (there it is again please laugh everyone!).
Initialization: On initialization, I1, I2, I3, I4 are all vacuous, since there is nothing to the right of $A[r]$ and nothing is to the left of $A[w]$.
Preserved: We perform a swap, so I1, I3 are temporarily broken, but we fix this before the end of this iteration so I1 and I3 remain true. Our swap before incrementing, as well as our loop guard $w \leq r$ ensure I2, I4 remain true.
Correctness: Once we terminate, we know our guard is false, that is $w > r$. At that point, I1, I2, I3, I4 are all satisfied, so we've reached our goal.
$\mathbb{Q}$: At the end, how are w and r related? $\implies$ $w = r + 1$. This implies that our algorithm **always** terminates! Note that proving this algorithm terminates is quite important and a very nontrivial step. □

Runtime

Proof. w is incremented by $+1$ and r is incremented by -1 for at most n steps, so we have upper bound $O(n)$. As for our lower bound: $r - w = n + 1$ at the start, and at the end $w = r + 1$ $\implies$ $\Omega(n)$ lower bound runtime. Thus we have a tight bound of $\Theta(n)$. □

4.3 Reductions

Definition 1: Reduction

A reduction from problem P to problem Q is an algorithm that solves P using (any algorithm for) Q. That is, we can use Q to solve P. We express this as $P \leq Q$ "P reduces to Q.
A nice way to think about this is taking an input x, (implicitly) transforming its input to Q, then transforming the output to $P(x)$.
Note that neither problem need be solvable!

- Squaring $\leq$ multiply: $x^2 = x \times x$
- And the other way around: multiply $\leq$ square: $(x + y)^2 - (x - y)^2$
- Median $\leq$ Sorting: sorted $(A)[\lfloor |A|/2 \rfloor]$

4.3.1 Types of Reduction: Turing/Oracle and Many-one

Definition 2: Turing Reduction

We denote this as $\leq_T$. We can use Q any number of times. Formally, a Turing Reduction from a problem P to problem Q is an algorithm A that solves P while using a subroutine solving Q as an atomic operation.

Definition 3: Many-one/Mapping Reduction

We denote this as $\leq_m$. A many-one reduction from problem P to problem Q is an algorithm (or function) f with the property that $P = Q \circ f$ or $P = Q(f(x))$ for input x. A many- one reduction is a Turing reduction with the restriction that the subroutine for Q can be used only once, at the ennd, and the answer **must be returned without modification**.
In other words, take an input x for P, transform it using f into an input $y = f(x)$ for Q, and then compute $Q(y)$, which should give the same answer as $P(x)$. A Many-One Reduction is necessarily a Turing Reduction.

- Square $\leq$ mult: This is a turing reduction, since we use mult at least once. Since we only use mult once in $f(x) = x \times x$ this is also a many-one reduction.
- Mult $\leq$ square: We use square twice, so this is a turing reduction, but not a many-one reduction.
- Median $\leq$ Sort: It is a turing reduction, since we use it once, but it's not a many-one reduction for an odd reason: **the output type is different**!

4.3.2 Median Reduces to Selection (Many-One)

Assume for the input $A, |A|$ odd and all elements distinct.
A first instinct would be to recurse, perhaps dividing the array into three and recursing on the middle partition, but that actually doesn't work, because the median of our subarray is not necessarily the median of our larger array. Instead, let's reduce this to Selection.

Selection Algorithm

- input: array A with distinct elems and an integer $k : 0 \leq k < |A|$.
- output: kth smallest elem of A.

For our selection algorithm, let's try the following. We guess an element $P \in A$ and check if that is our kth smallest by colouring all smaller elements red and running Polish Flag. When Polish terminates, we have 3 cases:

- $w = k$: We have found our median, return P.
- $w > k$: Recurse on the left (red) side: SELECT$(A[: w], k)$. We know our kth smallest is in the smaller half of our array, so we can discard the upper half.
- $w < k$: SELECT$(A[w + 1 :], k - w)$. We know our kth smallest element is in the upper half of our array, so we can discard the lower half and increment k by w to adjust for our new subarray.

Now, how do we choose P? A fixed (determinisitic) fast method for choosing P implies lower bound $\Omega(n^2)$, since the fixed algorithm may have some worsr case input for which it is ill-suited. Instead, let's choose **randomly**!

4.4 Randomisation

4.4.1 Quick Select

- Base case: if $|A|$ small, just sort and return the index
- Choose random $i \in (0, |A|)$; set $P = A[i]$
- Colour the smaller elements red; run Polish Flag
- If $w = k$: output P
- If $w > k$: QUICKSELECT$(A[: w], k)$
- If $w < k$: QUICKSELECT$(A[w + 1 :], w - k)$

Why can we only claim partial correctness? Exercise: Modify the algorithm slightly to make it correct (What are we talking about here?)

Definition 4: Expected Runtime

The expected runtime of a randomized algorithm is the maximum (over all inputs x of length n) of the expected (over the randomness of our algorithm) number of steps that our algorithm executes for input x. Caution: We're looking at the worst-case input here, not a random input! Ensure to **quantify** the input first and then **take the expectation of the algorithm's runtime**.
Note that the expected runtime is not the same as the average case runtime.

Let's prove the expected runtime of Quick select.

Proof. Let pivot be the ith smallest element in $\{0, ..., n-1\}$. Our expected runtime is the maximum number of steps (over k) that our algorithm executes. Now, when we choose a k, we have two cases: either our k too large and we recurse on the ones before k, or our k too small and we recurise on the ones after k. Note also that the probability of choosing this k for which we have this maximum number of steps is $\frac{1}{n}$. Asymptotically, these sums are identical, so we can do the following

$$\begin{aligned} T(n) &\leq \max_k \left[\frac{1}{n} \sum_{i=0}^{k-1} T(n-i) + \frac{1}{n} \sum_{i=k+1}^{n-1} T(i) + \Theta(n) \right] \\ &\leq \frac{2}{n} \left(\sum_{i=n/2+1}^{n} T(i) \right) + \Theta(n) \end{aligned}$$

Now, consider the pivot position (after Polish flag) landing in one of three regions:

- 0 to $n/4$: probability $1/4$
- $n/4 + 1$ to $3n/4$: probability $1/2$
- $3n/4 + 1$ to $n - 1$: probability $1/4$

The first and last regions are pretty bad regions for our pivot to fall, since if we get unlucky we then have to consider at most n elements on our next recursive step. The middle region is pretty good, since if we get unlucky we only have to recurse on $3n/4$ elements on our next step. So, (considering our next recursive step) with probability $1/2$ we're in the $T(n)$ region and with probability $1/2$ we're in the $T(3n/4)$ region.

$$\begin{aligned} T(n) &\leq \frac{1}{2}T(n) + \frac{1}{2}T(3n/4) + \Theta(n) \\ &\leq T(3n/4) + \Theta(n) \end{aligned}$$

□

5 Lecture 5: Sequences

Definition 1: Data Structure

A data structure is a data store, plus a means of accessing and manipulating the stored data. Algorithms rely on data structures to compute with large amopunts of data and data structures are built on algorithms that tell them how to manipulate the data they store.

Definition 2: Interface

An interface is a mathematical sepcification for a data structure. It states the type of object(s) represented and the required behaviour of supported operations. An implementation of an interface describes how to represent the data in memory, together with algorithms to perform each of the required operations. Note that an interface may have several different implementations - **they are not the same thing**!

5.1 Stack Interface and its Implementations

A stack uses a **last in first out** access pattern via

- `push(`x`)`: stores object x on the top of the stack
- `peek()`: returns the object x currently at the top of the stack
- `pop()`: returns and deletes the object x currently at the top of the stack
- `is-empty()`: returns **TRUE** iff the stack has no objects

5.1.1 Linked List

- Pointer-based data structure
- Each item stored in a `node` which contains a pointer to the next node below it. Each `node` has two fields: `node.item` and `node.next`
- Maintains a pointer to the top node `head` in the stack
- Insertion and deletion from the top takes $\Theta(1)$ time since we can just re-link pointers
- Can peek and check emptiness by reading `head` $\implies$ $\Theta(1)$ time

5.1.2 Static Array

- Arrays are contiguous chunks of memory cells.
- We maintain a pointer `top` (this isn't REALLY a pointer, but it's functionally one) pointing to the first empty cell in the array. Initialize `top` to 0.
- `push(c)`: we set `A[top]=c` and increment `top`.
- `pop()`: we decrement `top` and then return `A[top]`
- `peek()`: same as `pop()` then increment `top` to maintain our array state
- `is-empty()`: return **TRUE** iff `top` greater than 0

5.1.3 Dynamic Array

- n is the number of items stored and $m \geq n$ is the capacity. A resize that copies n items is linear, one that doesn't resize is constant.

5.2 Amortized Analysis

We consider the worst-case runtime of a sequence of operations. Now, this is a pretty nontrivial topic, but I've found that the best explanation for me is the following amortized analysis of appending (`push(`x`)`) a sequence of values to a dynamic array.

Proving Amortized Constant Time for insertion into dynamic array sequence: Suppose we double the size of the array when full. That is, we reallocate the entire array in memory when $i = 2^k + 1$. Let's analyze the total cost of n insertions, then divide this total cost by n.

Proof. **What is the cost of the i-th operation?** Case 1: If we do not need to increase the size of the array, algorithm takes $O(1)$ unit of time. Case 2: If we need to increase the size of the array, then at this step we must:

1. allocate memory for the new array (constant)
2. copy over the 2^k elements from the old array to the new array (2^k running time)
3. free the memory storing the old array (constant)
4. append the new element (constant)

so this algorithm takes $2^k + 1$ running time. Let $t(i) :=$ the amount of running time.

$$t(i) = \begin{cases} 2^k + 1, \text{ if } i = 2^k + 1 \\ 1 \text{ otherwise} \end{cases}$$

So we just sum over n.

$$\begin{aligned}\sum_{i=1}^{n} t(i) &= n + \sum_{k=0}^{\lfloor \log_2(n-1) \rfloor} 2^k \\ &= n + \frac{2^{\lfloor \log_2(n-1) \rfloor} - 1}{2-1} \\ &\leq n + 2n \\ &\leq 3n\end{aligned}$$

So the total cost of n insertions is at most $3n$, which has amortized cost at most 3. Thus, the amortized cost is $O(1)$. □

5.3 Memory Overhead

Definition 3: Memory Overhead

Memory overhead is the memory used in addition to our memory allocated for our data, as a function of the number of elements stored. Dynamic arrays have an overhead of $\Theta(n)$ since we may allocate space $2n$ when storing n elements (right after doubling). Linked list also have $\Theta(n)$ memory overhead, since in addition to the data stored in each node, we need to store a $\Theta(1)$ size memory address pointing to the next node in our list.

There's also the `SEQUENCE` interface, but I can't be bothered to write it here. You can implement all the previous implementations within the sequence interface, with some runtime advantages/tradeoffs and available constant time operations.

6 Lecture 6: Sets and Hashes

- `Build()` (Initialize Container)
- `size()` (Container)
- `Find(e)` (Static)
- `Insert(x) - replace if already exists` (Dynamic)
- `Delete(k)` (Order)

6.1 Comparison Model of Computation

We associate a key with each object/item stored and determine equivalence by objects' keys.
One way to ensure constant time lookup is to use a **direct access array**. Say we have an item with key k, our item is stored in the $k-th$ position in the sequence. Insertion and deletion also take constant time. However, we don't know how high our indexing goes. For example, if given 9 digit integer keys, we'd need to allocate 10^9 slots in memory to store the items. Moreover, the size of our largest key u is bounded by the word size w: $u < 2^w$ (this is strictly less than, because we're assuming $0 \leq k \leq 2^w - 1$).

What if we surpass this bound? We map a large space of u keys down to a range $m = \Theta(n)$. In other words, we define a map:

$$h : \{0, ..., u-1\} \to \{0, ..., n-1\}$$

that maps the big space of u things to a small space of $\Theta(n)$ things. By the pigeonhole principle, some of our items must collide after mapping. We have two options to deal with collisions:

1. **Open addressing:** over allocate our $\Theta(n)$ array and find another place to store it in there. (unimportant)
2. At that key, store a pointer to another data structure (like a sequence), where we can store a collection of items with collisions. That is, if our hashing function assigns x items the same key k, then at the corresponding position in our smaller array that we map to, we can point to another data structure storing the x items.

Recall that by the pigeonhole principle, collisions are inevitable with any hash function and for u very large, there is a possibility that all of our items collide. For now, we consider the easiest way to get from a large space of keys to a smaller one.

6.1.1 Division Method

$$h(k) = k\text{mod(m)}$$

This way, we take a key in the large space, and **mod** it so that it wraps around m spaces. In general, this distributes collisions uniformly given our keys are selected nicely. However, there still exists some collection of inputs that map everything to one place. In any case, for any fixed hash function, we can end up having all items collide (in the deterministic case). Instead, we can take a non-deterministic approach, where we choose a random hash function for any collection of keys input by the user.

6.1.2 Universal Hash Function

So, we define a family of hash functions and randomly choose from this collection. Start with a fixed hash function as shown below.

$$h_{ab} = (((ak + b) \text{ mod p}) \text{ mod m})$$

Now, generalize to a family:

$$H(p, m) = \{h_{ab}(k) | a, b \in \{0, ..., p-1\} \text{ and } a \neq 0\}$$

We have a hash family parameterized by the length of our smaller array m and fix some random large prime p bigger than u. When we instantiate the hash table, we choose a random a and b to generate our function. By doing this **non-deterministic** jumbling, it is very unlikely for any given collection of keys to have too frequent collisions. Using this **universal hash family**, for any two keys, the probability that they collide is less than $1/m$.

Define random variable X_{ij} over choice of $h \in H$, which means $X_{ij} = 1$ if $h(k_i) = h(k_j)$, 0 otherwise. Size of chain at $h(k_i)$ is just the sum:

$$\begin{aligned} X_i &= \sum_{j=0}^{n-1} X_{ij} \\ \mathbb{E}_{h \in H}[X_i] &= \mathbb{E}\left[\sum_j X_{ij}\right] \\ &= \sum_{j \neq i} \mathbb{E}[X_{ij}] + 1 \\ &= \sum_{j \neq i}^{n-1} (\frac{1}{m}) + 1 \\ &= \frac{n-1}{m} + 1 \end{aligned}$$

7 Lecture 7: Radix Sort

Trivial.

8 Lecture 8: Binary Search Trees

Binary trees are a generalization of linked lists where the tree is comprised of nodes that are connected to each other. Each node has up to two children below it, and up to one parent above it.

- The root of the tree is the unique node without a parent
- We access a binary tree data structure with a pointer to its root; in this way we can equate a binary tree with its root
- Each node in the tree is the root of the subtree rooted at the node
- A node is called a leaf if both its left and right children are empty. A leaf node has no descendants.
- The dept of a node is the number of its ancestors
- The height of a node is the length of the longest path from the node to a leaf in the subtree rooted at that node.
- The height of a tree is the height of its root; an empty tree has height -1.

8.1 Traversals

A common set of operations on trees are the so-called traversals. They enumerate all the nodes of a given tree in a specified order.

8.1.1 In-Order Traversals

Visits the left child of the root first, recursively, outputs the root node, and then visits the right child of the root recursively.

8.1.2 Pre-Order Traversal

The pre-order traversal outputs first the root node, then recursively traverses the left child, then recursively traverses the right child.

8.1.3 Post-Order Traversal

Recursively traverses the left child of the root, then recursively traverses the right child, and then outputs the root.

8.2 Binary Search Trees and Sets

We can implement the set interface using binary trees. To do this, suppose we have a collection of objects and each object x has a key k. We assume all keys are different. We can store the objects in the nodes of a binary tree and impose some structure to facilitate performing "binary searches" on the tree for values.
Search, insertion, deletion, min/max, and predecessor/successor quieries take $O(h)$ time where h is the tree height. Iteration takes $\Theta(n)$ and construction by sorting takes $\Theta(n \log n)$ time. We can also keep our tree semi-balanced such that $h \in \Theta(\log n)$.

9 Lecture 9: Augmented Binary Trees

9.1 Sequences as Binary Trees

Suppose we have a sequence $x_0, x_1, ..., x_n$ of elements. One way is to use the same construction as before, but with indices as keys. We can instead keep the indices **implicitly** through an augmented binary tree. Note that we will not be using the binary search property, since sequences don't have keys. Instead, we maintain the invariant that the in- order traversal of the tree outputs the sequence order. We call this tree a **binary-tree sequence**: its in-order traversal represents the sequence order, and its items need not satisfy the BST property.

9.2 Tree Augmentations

An **augmentation** stores extra information in a data structure so that it can support additional operations efficiently. It is required that

- the value only depends on the subtree
- the value can be computed efficiently in $O(1)$ time from the node's item and the values stored at the node's children
- a node's values do not depend on the values of its parent, or of any of its other ancestors

Another way to think of an augmentation is as a function $f(T)$ that takes an input tree and can be computed using only $T.item$, $f(T.left)$, and $f(T.right)$. For example, subtree size. This is just the sum of the size of the left and right children plus 1 (the node itself). A nonexample is the depth of a node, which requires looking at its ancestors, not the subtree.

Maintaining an Augmentation : Modifying trees may break augmentations, so we must repair the augmentations before we finish modifying the tree. A change to an augmentation in a node only propagates to the ancestors of that node in the tree (bottom-up).

9.3 Subtree Size: Order Statistics and Sequences

Order Statistics of a binary tree is about associating a node with its index or "rank" as derived from **in-order traversal** (first node has rank 1, and the last rank n). In a BST, the node of rank k holds the $k-th$ smallest key. In a binary-tree sequence, it holds the $k-th$ sequence element X_k.

9.4 Determining the Rank of an Element

Proof. The rank of x is the number of nodes preceding it in an in-order tree walk plus 1 for x itself. We count x's left subtree and x itself, then walk up toward the root. Moving up from a left child adds nothing (since we traverse the left first). Moving up from a right child adds the parent as well as the parent's left subtree. (more on this later if I feel up to it) □

10 Key Formulae and Theorems... Thanks ChatGPT

10.1 Theorems to Know

10.1.1 Master's Theorem

For a recurrence of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \qquad a \geq 1,\ b > 1,$$

let

$$c = \log_b a.$$

Then:

$$T(n) = \begin{cases} \Theta(n^c), & f(n) = O(n^{c-\epsilon}) \text{ for some } \epsilon > 0, \\ \Theta(n^c \log n), & f(n) = \Theta(n^c), \\ \Theta(f(n)), & f(n) = \Omega(n^{c+\epsilon}) \text{ for some } \epsilon > 0 \text{ and } af(n/b) \leq cf(n) \text{ for some } c < 1. \end{cases}$$

10.1.2 Fermat's Little Theorem

If p is prime and $a \not\equiv 0 \pmod{p}$, then

$$a^{p-1} \equiv 1 \pmod{p}.$$

Equivalently, for any integer a,

$$a^p \equiv a \pmod{p}.$$

10.1.3 Bezout's Identity

For integers a, b, there exist integers x, y such that

$$ax + by = \gcd(a, b).$$

In particular,

$$\gcd(a, b) = 1 \iff \exists x, y \in \mathbb{Z} \text{ such that } ax + by = 1.$$

10.1.4 Euclidean Algorithm

For integers a, b with $b > 0$,

$$\gcd(a, b) = \gcd(b, a \bmod b).$$

Repeatedly applying this identity gives the Euclidean algorithm.
Its running time is

$$O(\log \min(a, b)).$$

10.1.5 Extended Euclidean Algorithm

The extended Euclidean algorithm computes integers x, y satisfying

$$ax + by = \gcd(a, b).$$

If

$$\gcd(a, m) = 1,$$

then the coefficient x gives the modular inverse:

$$ax \equiv 1 \pmod{m}.$$

10.1.6 Euler's Theorem

If

$$\gcd(a, n) = 1,$$

then

$$a^{\varphi(n)} \equiv 1 \pmod{n},$$

where $\varphi(n)$ is Euler's totient function.

Fermat's Little Theorem is the special case

$$n = p$$

for prime p, since

$$\varphi(p) = p - 1.$$

10.1.7 Chinese Remainder Theorem

If

$$\gcd(n_i, n_j) = 1 \qquad \text{for all } i \neq j,$$

then the system

$$x \equiv a_1 \pmod{n_1}, \quad x \equiv a_2 \pmod{n_2}, \quad \ldots, \quad x \equiv a_k \pmod{n_k}$$

has a unique solution modulo

$$N = \prod_{i=1}^{k} n_i.$$

10.1.8 Binomial Theorem

For any integer $n \geq 0$,

$$(x + y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k.$$

The binomial coefficient is

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}.$$

10.1.9 Pascal's Identity

For $0 < k < n$,

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}.$$

Boundary cases:

$$\binom{n}{0} = \binom{n}{n} = 1.$$

10.1.10 Handshaking Lemma

For any undirected graph $G = (V, E)$,

$$\sum_{v \in V} \deg(v) = 2|E|.$$

Consequently, the number of vertices of odd degree is even.

10.1.11 Pigeonhole Principle

If $n+1$ objects are placed into n boxes, then at least one box contains at least two objects.

More generally, if N objects are placed into k boxes, then some box contains at least

$$\left\lceil \frac{N}{k} \right\rceil$$

objects.

10.1.12 Inclusion–Exclusion Principle

For two sets,

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

For three sets,

$$\begin{aligned} |A \cup B \cup C| = {} & |A| + |B| + |C| \\ & - |A \cap B| - |A \cap C| - |B \cap C| \\ & + |A \cap B \cap C|. \end{aligned}$$

In general,

$$\left| \bigcup_{i=1}^{n} A_i \right| = \sum_{\emptyset \neq S \subseteq [n]} (-1)^{|S|+1} \left| \bigcap_{i \in S} A_i \right|.$$

10.1.13 Cauchy–Schwarz Inequality

For real numbers $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$,

$$\left(\sum_{i=1}^{n} a_i b_i \right)^2 \leq \left(\sum_{i=1}^{n} a_i^2 \right) \left(\sum_{i=1}^{n} b_i^2 \right).$$

10.1.14 Triangle Inequality

For real numbers,

$$|a + b| \leq |a| + |b|.$$

More generally,

$$\left| \sum_{i=1}^{n} a_i \right| \leq \sum_{i=1}^{n} |a_i|.$$

10.1.15 Law of Total Probability

If $B_1, \ldots, B_n$ form a partition of the sample space, then

$$P(A) = \sum_{i=1}^{n} P(A \mid B_i) P(B_i).$$

10.1.16 Bayes' Theorem

For events A and B with $P(B) > 0$,

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}.$$

More generally, if $B_1, \ldots, B_n$ form a partition,

$$P(B_j \mid A) = \frac{P(A \mid B_j)P(B_j)}{\sum_{i=1}^{n} P(A \mid B_i)P(B_i)}.$$

10.1.17 Linearity of Expectation

For random variables $X_1, \ldots, X_n$,

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i].$$

This holds whether or not the random variables are independent.

10.1.18 Markov's Inequality

For a nonnegative random variable X and $a > 0$,

$$P(X \geq a) \leq \frac{\mathbb{E}[X]}{a}.$$

10.1.19 Chebyshev's Inequality

For a random variable X with finite mean μ and variance σ^2,

$$P(|X - \mu| \geq t) \leq \frac{\sigma^2}{t^2}.$$

Equivalently,

$$P(|X - \mu| \geq k\sigma) \leq \frac{1}{k^2}.$$

10.1.20 Union Bound

For any events $A_1, \ldots, A_n$,

$$P\left(\bigcup_{i=1}^{n} A_i\right) \leq \sum_{i=1}^{n} P(A_i).$$

10.1.21 Geometric Distribution

If independent trials have success probability p, then the probability that the first success occurs on trial k is

$$P(X = k) = (1 - p)^{k-1} p.$$

Its expectation is

$$\mathbb{E}[X] = \frac{1}{p}.$$

10.1.22 Birthday Bound

For k uniformly random choices from a set of size n, the probability of at least one collision is approximately

$$1 - e^{-k(k-1)/(2n)}.$$

In particular, collisions become likely when

$$k = \Theta(\sqrt{n}).$$

10.2 Summation Formulae

10.2.1 Basic Summations

$$\sum_{i=1}^{n} 1 = n$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2 = \Theta(n^4)$$

$$\sum_{i=1}^{n} i^k = \Theta(n^{k+1}) \qquad (k > -1)$$

10.2.2 Arithmetic Series:

$$\sum_{i=0}^{n-1} (a + id) = \frac{n}{2}\left(2a + (n-1)d\right)$$

10.2.3 Geometric Series:

$$\sum_{i=0}^{n-1} r^i = \frac{1 - r^n}{1 - r} \qquad (r \neq 1)$$

$$\sum_{i=0}^{\infty} r^i = \frac{1}{1 - r} \qquad (|r| < 1)$$

$$\sum_{i=0}^{n} r^i = \Theta(r^n) \qquad (r > 1)$$

10.2.4 Harmonic Series:

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n)$$

More precisely,

$$H_n = \ln n + \gamma + O(1/n)$$

$$\sum_{i=1}^{n} \frac{1}{i^p} = \begin{cases} \Theta(1), & p > 1, \\ \Theta(\log n), & p = 1, \\ \Theta(n^{1-p}), & 0 \le p < 1. \end{cases}$$

10.2.5 Logarithmic Summations:

$$\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n)$$

$$\sum_{i=1}^{n} \log(i) = \Theta(n \log n)$$

$$\sum_{i=1}^{n} \frac{\log i}{i} = \Theta((\log n)^2)$$

10.2.6 Powers of Two:

$$\sum_{i=0}^{\log_2 n} 2^i = 2^{\log_2 n + 1} - 1 = 2n - 1 = \Theta(n)$$

$$\sum_{i=0}^{\log_2 n} i = \Theta((\log n)^2)$$

$$\sum_{i=0}^{\log_2 n} \frac{1}{2^i} = \Theta(1)$$

$$\sum_{i=0}^{\log_2 n} \frac{n}{2^i} = \Theta(n)$$

10.2.7 Telescoping Sums:

$$\sum_{i=1}^{n} (a_i - a_{i+1}) = a_1 - a_{n+1}$$

A common example:

$$\sum_{i=1}^{n} \frac{1}{i(i+1)} = \sum_{i=1}^{n} \left(\frac{1}{i} - \frac{1}{i+1} \right) = 1 - \frac{1}{n+1} = \Theta(1)$$

10.2.8 Useful Rules:

$$\sum_{i=1}^{n} \Theta(f(i)) = \Theta\left(\sum_{i=1}^{n} f(i)\right)$$

$$\sum_{i=1}^{n} c\, f(i) = c \sum_{i=1}^{n} f(i)$$

$$\sum_{i=1}^{n} (f(i) + g(i)) = \sum_{i=1}^{n} f(i) + \sum_{i=1}^{n} g(i)$$

$$\sum_{i=1}^{n} f(i) \leq n \max_{1 \leq i \leq n} f(i)$$

$$\sum_{i=1}^{n} f(i) \geq n \min_{1 \leq i \leq n} f(i)$$

10.2.9 Integral Approximation:

For nonnegative increasing f,

$$\int_{1}^{n} f(x)\, dx \leq \sum_{i=1}^{n} f(i) \leq f(n) + \int_{1}^{n} f(x)\, dx$$

For nonnegative decreasing f,

$$\int_{1}^{n+1} f(x)\, dx \leq \sum_{i=1}^{n} f(i) \leq f(1) + \int_{1}^{n} f(x)\, dx.$$